

Trabajo Práctico: Autenticación OAuth2 Google en NestJS

Enunciado Oficial y Resumen del Proyecto

1. Descripción del Trabajo Práctico

El objetivo de este trabajo práctico es diseñar, estructurar e implementar un microservicio de backend utilizando el framework **NestJS** que resuelva el flujo completo de registro y autenticación de usuarios mediante el proveedor de identidad federada **Google OAuth2**. El sistema final debe ser capaz de procesar el inicio de sesión del usuario, persistir su información de forma híbrida y retornar un JSON Web Token (JWT) firmado de forma segura para las peticiones subsecuentes.

2. Objetivos de Aprendizaje

- Comprender e integrar el flujo de autorización de OAuth 2.0 y el intercambio seguro de credenciales con Google Identity.
- Implementar la arquitectura modular de NestJS utilizando controladores, servicios y módulos especializados.
- Configurar estrategias de autenticación reutilizables mediante Passport.js en un entorno TypeScript.
- Modelar y persistir registros de usuarios federados usando Prisma ORM y una base de datos relacional.
- Diseñar mecanismos de seguridad basados en sesiones sin estado utilizando firmas JWT.

3. Requerimientos Técnicos del Proyecto

Configuración en Google Cloud Console

El alumno deberá dar de alta un proyecto en la consola de Google Cloud, configurar la pantalla de consentimiento de OAuth, definir las credenciales de cliente web y registrar de forma precisa las URIs de redireccionamiento (callback) autorizadas.

Persistencia de Datos (Prisma ORM)

El modelo de base de datos debe contemplar una estrategia híbrida que permita registrar usuarios locales (email/password) y usuarios federados (Google). Se debe almacenar el ID de Google, nombre, email, avatar y la marca de tiempo de creación.

Lógica de Autenticación (NestJS + Passport)

Se debe implementar una estrategia extendiendo 'PassportStrategy' de '@nestjs/passport'. El método 'validate' de la estrategia de Google tiene que procesar el perfil devuelto por Google, buscar al usuario en la base de datos o crearlo si no existe, y pasar el objeto resultante al flujo de NestJS.

Emisión de Token JWT

Una vez validado el callback de forma exitosa, el controlador de autenticación invocará un servicio para firmar un token JWT con la información esencial del usuario. Este token será el encargado de proteger el acceso a las rutas privadas de la API.

4. Rúbrica de Evaluación

Criterio de Evaluación	Descripción / Requisito Mínimo	Puntaje
Configuración inicial y OAuth	Configuración correcta del módulo de variables de entorno, credenciales de cliente en Google Cloud Console y URI de redirección.	25 pts
Estrategia y Arquitectura	Implementación limpia de GoogleStrategy, AuthController y AuthService respetando los estándares de modularidad de NestJS.	30 pts
Base de Datos y Prisma	Modelado de datos robusto con Prisma ORM que soporte correctamente registros federados sin duplicar cuentas por email.	25 pts
Seguridad y Firma JWT	Generación adecuada del JSON Web Token tras un login exitoso, protegiendo adecuadamente las claves privadas de firma.	20 pts
